# Algorithms: An Analytical Note

Discrete Mathematics — Assignment

**Student:** Solanki Ruchit M. (ID: 202612085)
**Course:** MSc IT
**Subject:** Discrete Mathematics
**Instructor:** Prof. Gopinath Panda
**Reference:** K. H. Rosen, *Discrete Mathematics and Its Applications*, 8th ed., Chapter 3 (Algorithms)

**Abstract**

This note presents a complete, self–contained treatment of *algorithms* as developed in Chapter 3 of Rosen's *Discrete Mathematics and Its Applications*. It answers four questions in order: **(i)** what exactly an algorithm is and which properties make a procedure an algorithm; **(ii)** the principal *types* of algorithms (searching, sorting, greedy and divide–and–conquer); **(iii)** how the *growth of functions* is measured formally with the $\mathcal{O}$, $\Omega$ and $\Theta$ notations; and **(iv)** how *complexity* is analysed in the worst, average and best cases, together with the notion of tractability. Every definition is stated formally and illustrated with a small diagram, and each claim about running time is justified analytically. The companion artefacts of this submission are the LaTeX pseudocode file, the executed Jupyter notebook and the interactive HTML dashboard.

## 1. Introduction: What is an Algorithm?

The word *algorithm* derives from the ninth–century Persian mathematician Muhammad ibn Musa *al–Khwarizmi*; the Latinised form *Algoritmi* became *algorithm*. Intuitively an algorithm is a recipe, but discrete mathematics demands a precise definition.

**Definition 1.1** (Algorithm, Rosen §3.1)**.** An **algorithm** is a finite sequence of precise instructions for performing a computation or for solving a problem.

**Definition 1.2** (The seven properties)**.** A procedure is an algorithm only if it satisfies all of the following:

**(1) Input** — it receives a finite set of values from a specified set;

**(2) Output** — it produces a solution from a specified set;

**(3) Definiteness** — each step is precisely and unambiguously stated;

**(4) Correctness** — it produces the correct output for every legal input;

**(5) Finiteness** — it produces the output after a *finite* number of steps;

**(6) Effectiveness** — each step is basic enough to be carried out exactly;

**(7) Generality** — it solves a whole *class* of problems, not one instance.

Figure 1 summarises this view of an algorithm as a transducer of inputs into outputs.

### 1.1 Pseudocode conventions

Algorithms in Chapter 3 are written in *pseudocode*, an informal but unambiguous mixture of English and programming syntax. Its conventions are: assignment `:=`; block structure by indentation; control flow with `if--then--else`, `while`, `for`; procedures returning a value via `return`; and `{ }` for a comment. Because pseudocode hides the machine, it exposes the *logic* and therefore the *cost* of a procedure — the central concern of §3 and §4 below.

### 1.2 The limits of algorithms

Not every problem has an algorithmic solution. The **halting problem** asks for a procedure that decides, for an arbitrary program and input, whether the program terminates. Turing proved that *no such algorithm exists*: the problem is *undecidable*. This is the canonical reason the definition above insists on *definiteness* and *finiteness* — an algorithm is not merely "a way of computing", but a guaranteed termination with a specified answer.

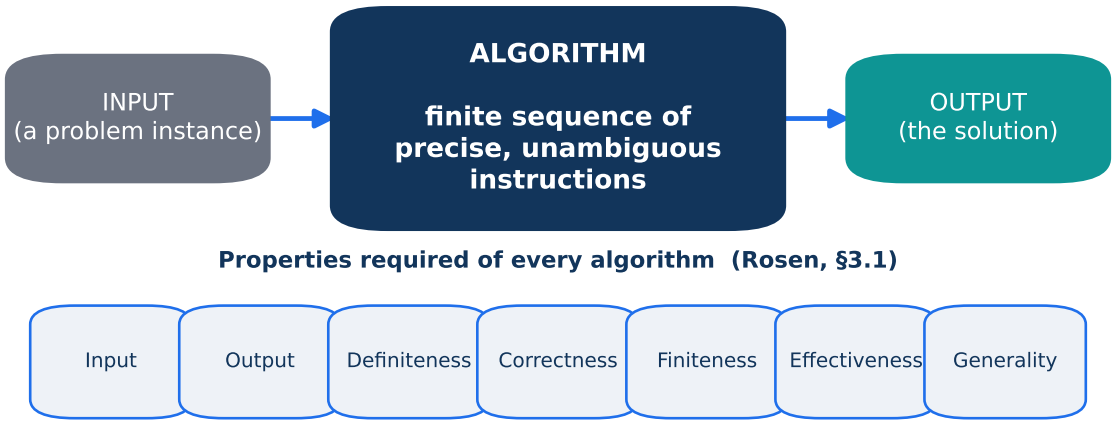



**Figure 1:** An algorithm transforms an input instance into an output solution, and must satisfy the seven properties of Definition 1.2.

## 2. Types of Algorithms

Chapter 3 presents several *paradigms*. A paradigm is a strategy for designing an algorithm, and the paradigm usually determines the growth rate of the resulting procedure.

### 2.1 Searching algorithms

**Linear search** examines the list elements one after another until the target is found or the list is exhausted. **Binary search** works only on a list that is *already sorted*: it compares the target with the middle element and discards half of the remaining list at every step. The two behave completely differently, as Figure 2 shows.

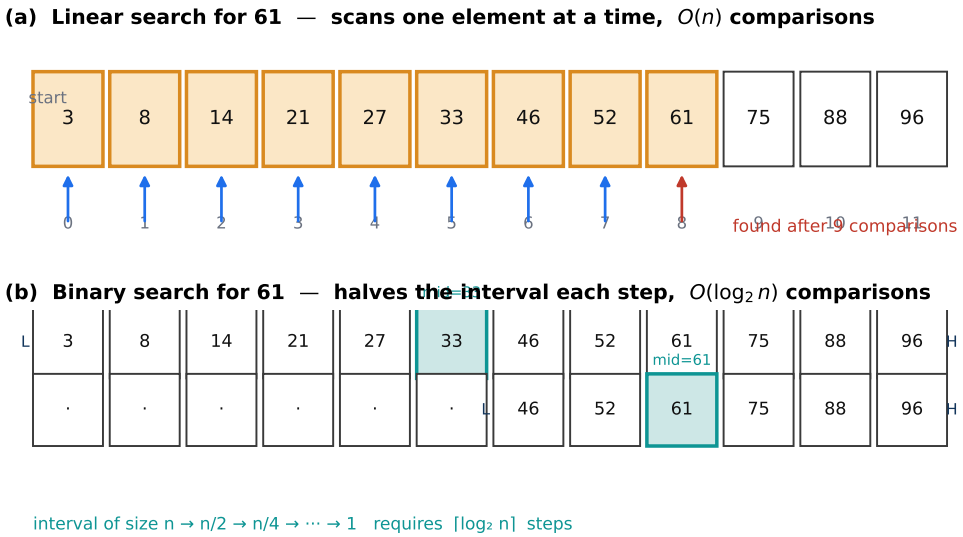



**Figure 2:** Searching for 61 in a sorted list of 12 elements. (a) Linear search performs 9 comparisons; (b) binary search performs only 3, because each step halves the interval.

### 2.2 Sorting algorithms

Sorting rearranges a list into increasing order. Three elementary $\mathcal{O}(n^2)$ sorts are treated in Chapter 3:

- **Bubble sort** repeatedly compares adjacent pairs and swaps them when out of order; each pass moves the largest remaining value to the end (Figure 3).
- **Insertion sort** maintains a sorted prefix and inserts the next element into its correct place.
- **Selection sort** repeatedly selects the smallest remaining element and appends it to the sorted

prefix.

A faster divide–and–conquer sort, **merge sort**, splits the list in half, sorts each half recursively and merges them in linear time, giving $\mathcal{O}(n \log n)$.

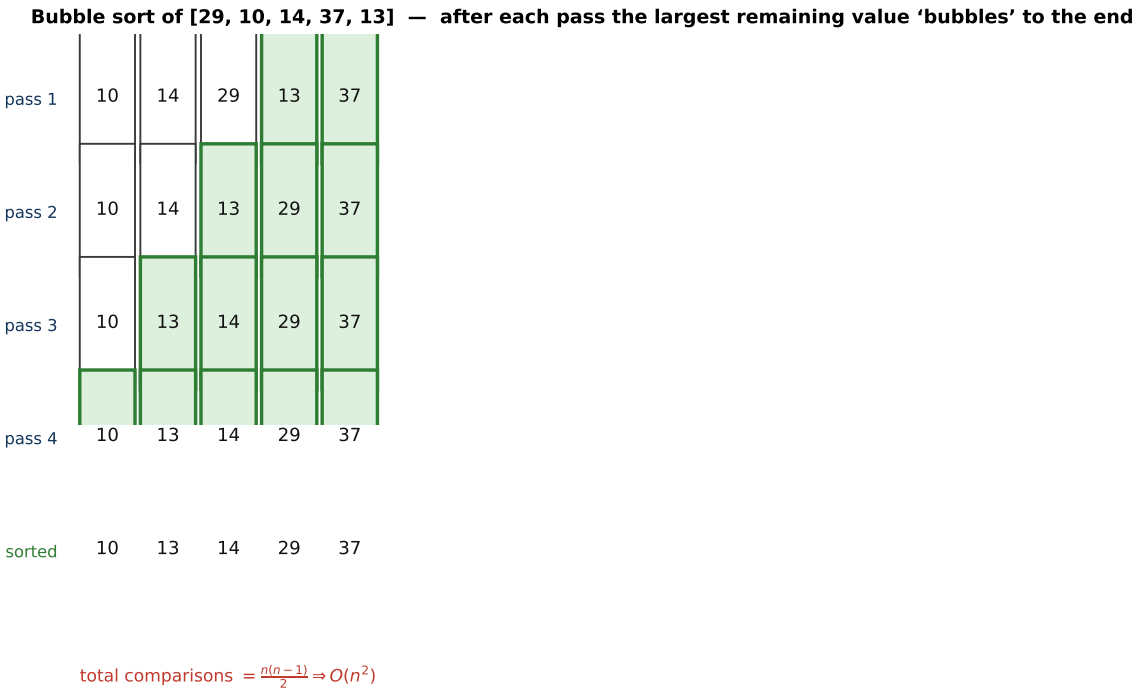



**Figure 3:** Bubble sort applied to $[29, 10, 14, 37, 13]$. Shaded cells are already in final position. The number of comparisons is exactly $n(n-1)/2$, hence $\mathcal{O}(n^2)$.

### 2.3 Greedy algorithms

A **greedy algorithm** builds a solution one step at a time, always choosing the option that looks best *right now*. The classical illustration is *change–making*: to give change of $n$ cents using $25, 10, 5, 1$–cent coins, repeatedly take the largest coin not exceeding the remaining amount. Greedy is fast (one pass over the denominations) but it is *not always optimal* for arbitrary coin systems — the U.S. system is one of the lucky ones.

### 2.4 Recursive and divide–and–conquer algorithms

A **recursive** algorithm calls itself on a smaller instance. When the recursion splits the input into independent parts, the paradigm is called *divide–and–conquer*; its cost is described by a *recurrence relation*. Merge sort satisfies $T(n) = 2T(n/2) + n$, whose solution is $\Theta(n \log n)$.

**Table 1:** Summary of the algorithm types discussed in Chapter 3.

| Type | Idea | Typical example | Growth |
|---|---|---|---|
| Linear search | scan every element | find a key in an unsorted list | $\mathcal{O}(n)$ |
| Binary search | halve a sorted interval | find a key in a sorted list | $\mathcal{O}(\log n)$ |
| Bubble / insertion / selection sort | repeated swaps or insertions | sort a small list | $\mathcal{O}(n^2)$ |
| Merge sort | divide, conquer, merge | sort a large list | $\mathcal{O}(n \log n)$ |
| Greedy | make the locally best choice | coin change, scheduling | $\mathcal{O}(n)$–$\mathcal{O}(n \log n)$ |
| Recursive | reduce to a smaller instance | Towers of Hanoi, factorials | $2^n$, $n!$, … |

## 3. The Growth of Functions

To compare algorithms we must compare the *growth* of the functions that count their operations as the input size $n$ grows. Chapter 3 formalises this intuition with the "big–$\mathcal{O}$" family of notations.

**Definition 3.1** (Big–$\mathcal{O}$)**.** Let $f, g : \mathbb{Z} \to \mathbb{R}$. We say $f(x)$ is $\mathcal{O}(g(x))$ if there exist *witnesses* $C > 0$ and $k > 0$ such that

$$|f(x)| \le C\,|g(x)| \qquad \text{for all } x > k.$$

**Definition 3.2** (Big–$\Omega$ and Big–$\Theta$)**.** $f(x)$ is $\Omega(g(x))$ if $|f(x)| \ge C\,|g(x)|$ for all $x > k$ (a lower bound). We write $f(x)$ is $\Theta(g(x))$ if $f(x)$ is both $\mathcal{O}(g(x))$ and $\Omega(g(x))$, i.e. $f$ and $g$ grow at the same rate.

The witnesses $C$ and $k$ are the whole content of the definition: $\mathcal{O}$ makes a claim about the *tail* of the function, not about small values.

**Theorem 3.1** (Sum and product rules)**.** *If $f_1 = \mathcal{O}(g_1)$ and $f_2 = \mathcal{O}(g_2)$, then*

$$(f_1 + f_2)(x) = \mathcal{O}\big(\max(|g_1(x)|, |g_2(x)|)\big), \qquad (f_1 f_2)(x) = \mathcal{O}\big(g_1(x) g_2(x)\big).$$

**Theorem 3.2** (Polynomials)**.** *If $a_0, a_1, \ldots, a_k$ are real with $a_k \neq 0$, then*

$$a_k n^k + a_{k-1} n^{k-1} + \cdots + a_0 \;=\; \mathcal{O}(n^k),$$

*so the leading term alone determines the growth of a polynomial.*

Figure 4 plots the standard functions. On a linear scale the exponential $2^n$ destroys every polynomial; on a logarithmic scale the powers $n$, $n^2$, $n^3$ become straight lines, which is exactly why logarithmic plots are used to compare growth rates.

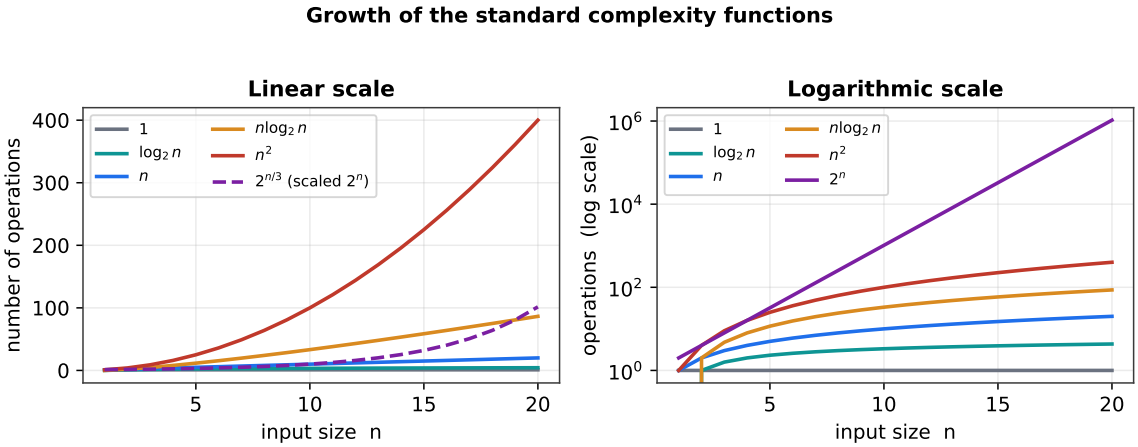



**Figure 4:** Growth of the standard complexity functions on a linear axis (left) and a logarithmic axis (right).

**Table 2:** Numerical values of the standard growth functions.

| $n$ | $\log_2 n$ | $n$ | $n \log_2 n$ | $n^2$ | $2^n$ |
|---|---|---|---|---|---|
| 10 | 3.3 | 10 | 33 | $10^2$ | $1.02 \times 10^3$ |
| $10^2$ | 6.6 | $10^2$ | $6.6 \times 10^2$ | $10^4$ | $1.27 \times 10^{30}$ |
| $10^3$ | 10 | $10^3$ | $10^4$ | $10^6$ | $1.07 \times 10^{301}$ |
| $10^4$ | 13.3 | $10^4$ | $1.3 \times 10^5$ | $10^8$ | astronomically large |

## 4. Complexity of Algorithms

### 4.1 Time complexity

The **time complexity** of an algorithm is the number of basic operations it performs, expressed as a function of the input size $n$ using the notation of §3. Three cases are distinguished:

- **Worst–case** complexity $W(n)$ — the maximum over all inputs of size $n$; the most useful guarantee.
- **Average–case** complexity $A(n)$ — the expected cost assuming a probability distribution on inputs.
- **Best–case** complexity $B(n)$ — the minimum; usually of little practical value.

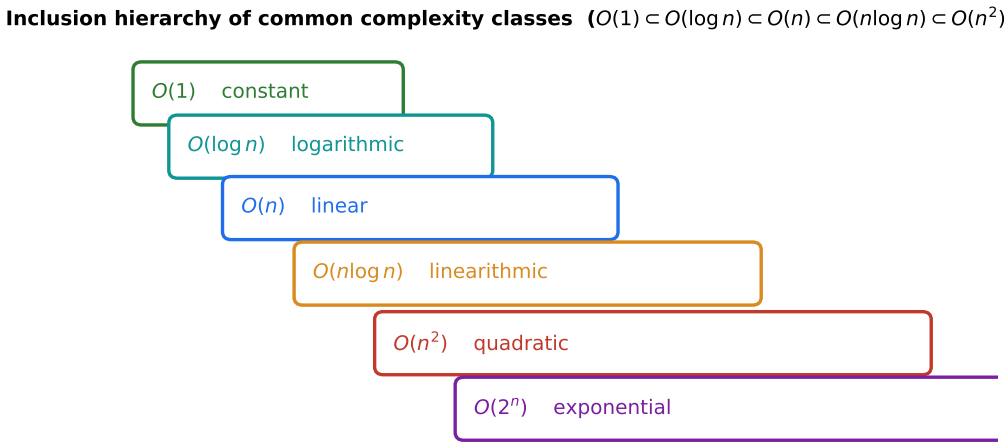



**Figure 5:** Inclusion hierarchy of the common complexity classes. Each class contains all the classes drawn inside it.

### 4.2 Worked complexity analysis

**Linear search.** In the worst case the target is absent and every element is tested; the cost is $\Theta(n)$.

**Binary search.** After $i$ steps the interval has size $n/2^{i-1}$; it becomes empty when $n/2^{i-1} < 1$, i.e. $i > \log_2 n + 1$. Hence $W(n) = \Theta(\log n)$ — the enormous saving visible in Figure 2.

**Bubble sort.** The inner loop performs comparisons

$$\sum_{i=1}^{n-1}(n-i) = \frac{n(n-1)}{2} = \Theta(n^2),$$

so bubble sort is quadratic in every case.

### 4.3 Space complexity and tractability

**Space complexity** counts memory cells rather than operations. A problem is called *tractable* if it can be solved by an algorithm with polynomial worst–case time complexity; problems requiring exponential time are deemed *intractable*, because for even modest $n$ the count explodes (Figure 6 and Table 2). The question of whether every problem whose solution can be *checked* quickly can also be *solved* quickly is the famous P versus NP problem.

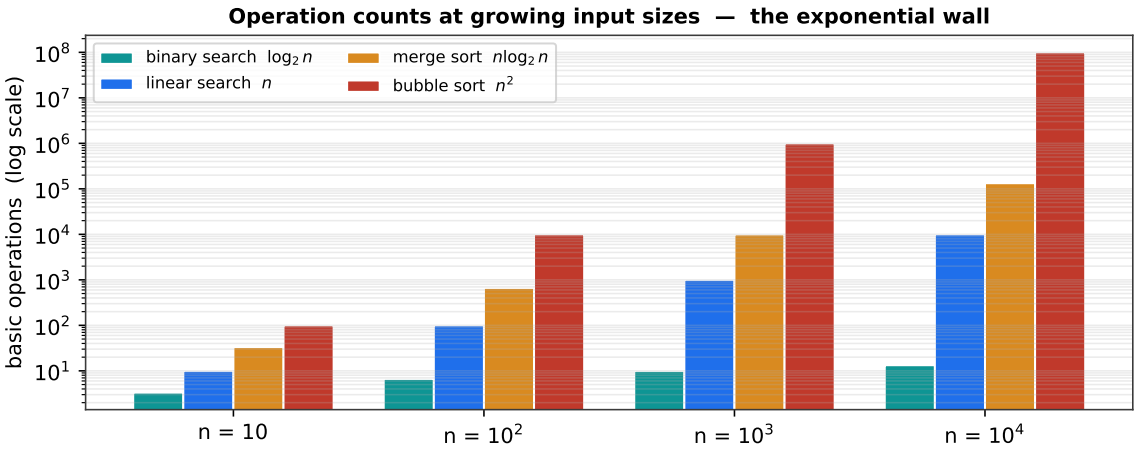



**Figure 6:** Basic–operation counts at growing input sizes. The polynomial methods stay feasible while the number of operations for exponential methods becomes astronomical.

## 5. Worked Analytical Examples

**Example 5.1** (Finding the witnesses)**.** Show that $f(n) = 3n^2 + 2n + 5$ is $\mathcal{O}(n^2)$.

*Proof.* For every $n \geq 1$ we have $2n \leq 2n^2$ and $5 \leq 5n^2$. Therefore

$$f(n) = 3n^2 + 2n + 5 \leq 3n^2 + 2n^2 + 5n^2 = 10n^2.$$

Taking $C = 10$ and $k = 1$ gives $f(n) \leq 10n^2$ for all $n > 1$, so $f(n) = \mathcal{O}(n^2)$. Moreover $f(n) \geq 3n^2$ for $n \geq 1$, so $f(n) = \Omega(n^2)$; combining the two bounds, $f(n) = \Theta(n^2)$. □

**Example 5.2** (Separating two growth rates)**.** Show that $n^2$ is *not* $\mathcal{O}(n \log n)$.

*Proof.* Suppose, for contradiction, that $n^2 \leq C\, n \log_2 n$ for all $n > k$. Dividing by $n \log_2 n$ gives

$$\frac{n}{\log_2 n} \leq C \qquad \text{for all } n > k.$$

But $n/\log_2 n \to \infty$ as $n \to \infty$; choosing $n > 2^{C+1}$ makes the quotient exceed $C$, a contradiction. Hence $n^2 \neq \mathcal{O}(n \log n)$, which is precisely why merge sort beats bubble sort on large inputs. □

**Example 5.3** (Binary search recurrence)**.** Let $W(n)$ be the worst–case number of comparisons of binary search. Then

$$W(n) = W(\lfloor n/2 \rfloor) + 1, \qquad W(1) = 1.$$

Unrolling the recurrence, $W(n) = 1 + \lfloor \log_2 n \rfloor$, i.e. $W(n) = \Theta(\log n)$.

## 6. Conclusion

An algorithm is a finite, definite, effective and general procedure whose correctness is guaranteed for every legal input. Its power is measured not by how clever it looks but by how its operation count *grows*: the $\mathcal{O}$–$\Omega$–$\Theta$ calculus of §3 turns "this algorithm feels faster" into a theorem, and the worst–case analysis of §4 decides whether a problem is tractable. The four paradigms of §2 illustrate the whole range: a single change of strategy — scanning versus halving, or quadratic versus divide–and–conquer — moves a procedure from $\mathcal{O}(n^2)$ to $\mathcal{O}(n \log n)$ or from $\mathcal{O}(n)$ to $\mathcal{O}(\log n)$, which for large inputs is the difference between feasible and impossible.

## References


[1] K. H. Rosen, *Discrete Mathematics and Its Applications*, 8th ed., McGraw–Hill, 2019. **Chapter 3: Algorithms** (pp. 191–242).

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, 2009.

[3] D. E. Knuth, *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*, 3rd ed., Addison–Wesley, 1997.

[4] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed., Addison–Wesley, 2011.

[5] J. Kleinberg and E. Tardos, *Algorithm Design*, Pearson/Addison–Wesley, 2006.

[6] A. M. Turing, "On computable numbers, with an application to the Entscheidungsproblem", *Proc. London Math. Soc.*, ser. 2, vol. 42, pp. 230–265, 1936.